

Addis Ababa Science and Technology University

Artificial Intelligence & Robotics Center of Excellence (AI&R CoE)

Deep Learning Lab Manual for MSc. AI Students

Course Code: ARIN6022

Author: Ewnetu Abebe (MSc.)

Editor 1: Habib Mohammed (PhD.)

Signature: _____

Date: _____

Editor 2: Habtamu Girum (MSc.)

Signature: _____

Date: _____

Approved By: Habib Mohammed (PhD.)

Signature: _____

Date: _____

Table of Contents

Program Learning Outcomes **3**

1 Artificial Intelligence Foundations **4**
1.1 Historical and Modern Definitions of Artificial Intelligence 4
1.2 Current Role of Machine Learning and Deep Learning 5
1.3 Fundamental Concepts of Probability in AI 7

2 Core Machine Learning Methods **11**
2.1 Generative and Discriminative Approaches 11
2.2 Perceptrons and Artificial Neural Networks 12
2.3 Support Vector Machines (SVM) and Kernel Functions 13
2.4 Principles of Active Learning 13

3 Deep Learning Architectures **18**
3.1 Convolutional Neural Networks (CNNs) 18
3.1.1 Motivation 18
3.1.2 Convolution Operation 18
3.1.3 Stride and Padding 18
3.1.4 Activation Function (ReLU) 19
3.1.5 Pooling Layer 19
3.1.6 CNN Architecture Diagram 19
3.1.7 Types of Gradient Descent 19
3.2 Stochastic Gradient Descent (SGD) 20
3.2.1 Gradient Descent Update Rule 20
3.2.2 SGD with Momentum 20
3.3 Batch Normalization 20
3.3.1 Batch Normalization Steps 20
3.3.2 Sample Code Template (PyTorch — Jupyter Notebook Style) . . 21
3.4 Comparative Analysis Tables 22
3.4.1 CNN vs Fully Connected Neural Networks 22
3.4.2 SGD vs Adam Optimizer 23
3.5 Laboratory Exercises 23
3.5.1 Lab 1: Implementing a CNN for Image Classification 23
3.5.2 Lab 2: Comparing Optimizers (SGD vs Adam) 24
3.5.3 Lab 3: Experimenting with Batch Normalization 24

4 Advanced Generative Models **26**
4.1 Generative Adversarial Networks (GANs) 26
4.2 Deterministic and Variational Autoencoders (VAEs) 28

4.3 Flow-based Models (e.g., PixelCNN, WaveNet) 29
4.4 Self-Supervised Learning Methods (SSL) 30

5 Sequential Models **36**
5.1 Recurrent Neural Networks (RNNs) 36
5.2 Long Short-Term Memory (LSTM) Networks 37
5.3 Gated Recurrent Units (GRUs) 38

6 Learning Theory and Challenges **43**
6.1 Theoretical Foundations of Learning 43

Learning Outcomes

- Apply advanced mathematical and computational foundations.
- Design and implement machine learning and AI models.
- Analyze and evaluate AI systems critically.
- Conduct independent research and experimentation.
- Address ethical, bias, and societal issues in AI.

Chapter 1

Artificial Intelligence Foundations

1.1 Historical and Modern Definitions of Artificial Intelligence

Historical Definitions

Artificial Intelligence (AI) has evolved significantly since its inception. Early definitions focused on the ability of machines to mimic human intelligence.

- **1956, Dartmouth Conference:** AI was formally born as a field of study. The founders (McCarthy, Minsky, Rochester, and Shannon) defined AI as "the science and engineering of making intelligent machines".
- **1960s–1980s:** AI was often associated with symbolic reasoning and expert systems. Early researchers emphasized logical reasoning, rule-based systems, and problem-solving abilities that resembled human cognitive processes.
- **1980s Expert Systems Era:** AI systems were defined as tools that could encode human expertise into rules (e.g., MYCIN for medical diagnosis).

Modern Definitions

With advances in data, computing power, and algorithms, AI definitions have broadened:

- **Machine-Oriented:** AI is the capability of machines to perform tasks that, if done by humans, would require intelligence.
- **Task-Oriented:** AI systems can perceive their environment, reason, learn, and make decisions to achieve specific goals.
- **Examples:** Voice assistants (Siri, Alexa), autonomous vehicles, recommendation systems.
- **Key Points:** Modern AI focuses less on replicating human thought and more on solving practical problems effectively.

1.2 Current Role of Machine Learning and Deep Learning

Machine Learning (ML)

ML is a subset of AI that enables systems to learn patterns from data without explicit programming.
Key Concepts:

- **Supervised Learning:** Learning from labeled data (e.g., predicting house prices).
- **Unsupervised Learning:** Finding hidden patterns in unlabeled data (e.g., clustering customers).
- **Reinforcement Learning:** Learning through trial-and-error interactions with an environment (e.g., AlphaGo).

Role in AI:

- ML provides AI systems with the ability to improve automatically over time.
- It is widely used in image recognition, natural language processing, fraud detection, and predictive analytics.

Deep Learning (DL)

DL is a subset of ML based on artificial neural networks with multiple layers.
Key Concepts:

- **Neural Networks:** Networks of interconnected nodes (neurons) that can approximate complex functions.
- **Layers:** Input, hidden, and output layers allow hierarchical feature learning.
- **Applications:** Speech recognition, autonomous driving, medical imaging, language translation.

Relationship to AI:

- DL has enabled breakthroughs in AI due to its ability to process large-scale data and extract complex patterns.
- Modern AI systems heavily rely on DL for tasks requiring perception and cognition.

Summary Table:

Aspect	ML	DL	AI
Data requirement	Moderate	Large	Varies
Feature extraction	Manual	Automatic	Depends on approach
Complexity	Low to moderate	High	Varies
Applications	Predictive modeling	Image, speech, text processing	Depends on task

1.3 Fundamental Concepts of Probability in AI

Probability theory is central to AI for handling uncertainty, learning from data, and reasoning under incomplete information.

Random Variables

Variables that can take different values with certain probabilities.
Example: X = outcome of rolling a dice.

Probability Distributions

Describe how probabilities are assigned.
Discrete: $P(X = x)$, e.g., dice roll.
Continuous: $f(x)$, e.g., normal distribution of heights.

Conditional Probability

Probability of A given B has occurred:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

Bayes' Theorem

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}$$

Applications: spam detection, medical diagnosis, Bayesian networks.

Independence

Two events A and B are independent if:

$$P(A \cap B) = P(A)P(B)$$

Expectation & Variance

Expectation: $E[X] = \sum_x xP(X = x)$ (discrete)
Variance: $\text{Var}(X) = E[(X - E[X])^2]$

Applications in AI

- Handling uncertainty in predictions.
- Probabilistic models: Naive Bayes, Hidden Markov Models, Bayesian networks.
- Reinforcement learning relies on expected rewards for decision-making.

Iris Dataset Lab

Dataset

Dataset: Iris Flower Dataset (publicly available at UCI Machine Learning Repository).
Dataset Link: https://archive.ics.uci.edu/ml/datasets/iris
Dataset Description:

- 150 samples of iris flowers from 3 species: Setosa, Versicolor, Virginica.
- 4 features: Sepal Length, Sepal Width, Petal Length, Petal Width.
- Target variable: Species.

Lab Tasks

Task 1: Data Loading and Exploration

- Load the dataset using Python (Pandas or Scikit-learn).
- Display the first 10 rows.
- Check for missing values and basic statistics (mean, min, max, standard deviation).
- Visualize the distribution of features using histograms and boxplots.

Sample Code:

```python
import pandas as pd
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt
import seaborn as sns

# Load dataset
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target

# Display data
print(df.head())

# Check statistics
print(df.describe())

# Visualize
sns.pairplot(df, hue='species')
plt.show()
```

Task 2: Probability and Distribution Analysis

- Compute the probability of each species in the dataset.
- Compute mean and variance of each feature for each species.
- Visualize probability distributions using histograms.

Sample Code:

```python
# Probability of each species
prob_species = df['species'].value_counts(normalize=True)
print(prob_species)

# Mean and variance
grouped = df.groupby('species').agg(['mean','var'])
print(grouped)
```

Task 3: Implement a Simple Machine Learning Model

- Split the dataset into training (80%) and testing (20%) sets.
- Train a Naive Bayes classifier (probabilistic model) to classify iris species.
- Predict the test set and compute accuracy.

Expected Learning Outcomes

- Students will be able to explore and visualize datasets effectively.
- Students will understand how probability is used in classification tasks.
- Students will gain hands-on experience building a simple ML model.
- Students will develop critical thinking about model performance and limitations.

Chapter 2

Core Machine Learning Methods

2.1 Generative and Discriminative Approaches

Generative Models

- Model the **joint probability** $P(X, Y)$. - Can generate new data points.
Key Formula:

$$P(Y|X) = \frac{P(X|Y)P(Y)}{P(X)}$$

Example: Naive Bayes

$$P(Y|X) \propto P(Y) \prod_{i=1}^{n} P(x_i|Y)$$

Discriminative Models

- Model the **conditional probability** $P(Y|X)$ directly.
Example: Logistic Regression:

$$P(Y = 1|X) = \frac{1}{1 + e^{-(w^T X + b)}}$$

2.2 Perceptrons and Artificial Neural Networks

Perceptron

$$y = \begin{cases} 1 & \text{if } w^T X + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

Learning Rule:

$$w \leftarrow w + \eta(y_{\text{true}} - y_{\text{pred}})X, \quad b \leftarrow b + \eta(y_{\text{true}} - y_{\text{pred}})$$

Multi-Layer Perceptron (MLP/ANN)

Forward propagation:

$$a^{(l)} = f(W^{(l)} a^{(l-1)} + b^{(l)})$$

Backpropagation (gradient descent):

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial L}{\partial W^{(l)}}, \quad b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial L}{\partial b^{(l)}}$$

2.3 Support Vector Machines (SVM) and Kernel Functions

SVM

Decision function:

$$f(x) = w^T x + b$$

Hard margin optimization:

$$\min_{w,b} \frac{1}{2}||w||^2 \quad \text{s.t. } y_i(w^T x_i + b) \geq 1$$

Soft margin:

$$\min_{w,b,\xi} \frac{1}{2}||w||^2 + C\sum_{i=1}^{n} \xi_i, \quad y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

Kernel Functions

- **Linear:** $K(x, x') = x^T x'$
- **Polynomial:** $K(x, x') = (x^T x' + c)^d$
- **RBF/Gaussian:** $K(x, x') = \exp\left(-\frac{||x-x'||^2}{2\sigma^2}\right)$

2.4 Principles of Active Learning

- Reduce labeling cost by selecting **informative samples**.

- **Uncertainty sampling:**

$$x^* = \arg\max_x (1 - \max_y P_\theta(y|x))$$

- **Expected model change:** Select samples that change the model most.

$$\min_{\text{samples } S} E_{x \sim P(X)}[L(f(x), y)]$$

Lab Exercise: Core Machine Learning Methods

Lab Objectives

After completing this lab, students will be able to:

- Understand the difference between generative and discriminative models.
- Implement a simple Perceptron and Multi-Layer Perceptron (MLP) for classification.
- Train and evaluate a Support Vector Machine (SVM) classifier with linear and RBF kernels.
- Apply active learning strategies to reduce labeling effort.
- Visualize decision boundaries and analyze model performance.

Dataset

Iris Flower Dataset: 150 samples, 3 classes: Setosa, Versicolor, Virginica.
Features: Sepal Length, Sepal Width, Petal Length, Petal Width.
Target: Species.
Source: UCI Machine Learning Repository: Iris Data

Lab Steps

1. Data Preparation

- Load the Iris dataset using Python.
- Split into training (70%) and testing (30%) sets.
- Standardize features (mean=0, variance=1).

Python Example:

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Load dataset
iris = load_iris()
X, y = iris.data, iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Standardize features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

2. Generative vs Discriminative Models

- Implement Gaussian Naive Bayes (generative).
- Implement Logistic Regression (discriminative).
- Train both and compute accuracy; compare performance.

Python Example:

```python
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Gaussian Naive Bayes
gnb = GaussianNB()
gnb.fit(X_train, y_train)
y_pred_gnb = gnb.predict(X_test)
print("Generative Model Accuracy:", accuracy_score(y_test, y_pred_gnb))

# Logistic Regression
lr = LogisticRegression(max_iter=200)
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)
print("Discriminative Model Accuracy:", accuracy_score(y_test, y_pred_lr))
```

3. Perceptron and MLP

- Implement a Perceptron classifier.
- Implement an MLP with one hidden layer (ReLU activation).
- Train both and visualize decision boundaries (using 2 features).

Python Example:

```python
from sklearn.linear_model import Perceptron
from sklearn.neural_network import MLPClassifier

# Perceptron
perceptron = Perceptron(max_iter=1000, eta0=0.1, random_state=42)
perceptron.fit(X_train[:, :2], y_train)  # First 2 features

# MLP
mlp = MLPClassifier(hidden_layer_sizes=(5,), max_iter=1000, activation='relu', random_sta
mlp.fit(X_train[:, :2], y_train)
```

4. Support Vector Machines (SVM)

- Train linear and RBF-kernel SVMs.
- Evaluate accuracy and plot decision boundaries (2 features).

Python Example:

```python
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix

# Linear SVM
svm_linear = SVC(kernel='linear')
svm_linear.fit(X_train[:, :2], y_train)
y_pred_linear = svm_linear.predict(X_test[:, :2])
print("Linear SVM Accuracy:", accuracy_score(y_test, y_pred_linear))

# RBF SVM
svm_rbf = SVC(kernel='rbf')
svm_rbf.fit(X_train[:, :2], y_train)
y_pred_rbf = svm_rbf.predict(X_test[:, :2])
print("RBF SVM Accuracy:", accuracy_score(y_test, y_pred_rbf))
```

5. Active Learning (Optional/Advanced)

- Start with 20% labeled data and train a classifier.
- Select samples with highest uncertainty and label them.
- Repeat until accuracy stabilizes.

Python Hint:

```
# Select samples with maximum uncertainty
probs = lr.predict_proba(X_unlabeled)
uncertainty = 1 - probs.max(axis=1)
samples_to_label = X_unlabeled[uncertainty.argsort()[-5:]]  # top 5 uncertain samples
```

Expected Outcomes

- Compare performance of generative vs discriminative models.
- Observe how Perceptron fails on non-linear separable data.
- See MLP and SVM with RBF handle non-linear boundaries.
- Understand how active learning reduces labeled data while maintaining performance.
- Learn to visualize decision boundaries for classification.

Chapter 3

Deep Learning Architectures

3.1 Convolutional Neural Networks (CNNs)

Learning Outcomes

After completing these labs, students will be able to:

- Implement CNNs using PyTorch
- Compare deep learning architectures and optimizers
- Analyze experimental results critically

3.1.1 Motivation

Convolutional Neural Networks (CNNs) are designed to process data with a grid-like topology, such as images. They exploit **local connectivity**, **parameter sharing**, and **spatial hierarchies**.

3.1.2 Convolution Operation

2D Convolution

Let the input image be $X \in \mathbb{R}^{H \times W}$ and the kernel be $K \in \mathbb{R}^{k \times k}$. The convolution output is:

$$Y(i,j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X(i+m, j+n)\, K(m,n)$$

For multi-channel input:

$$Y(i,j) = \sum_{c=1}^{C} \sum_{m,n} X_c(i+m, j+n) K_c(m,n)$$

3.1.3 Stride and Padding

The output feature map size is:

$$\text{Output} = \left\lfloor \frac{N - K + 2P}{S} \right\rfloor + 1$$

3.1.4 Activation Function (ReLU)

ReLU

$$f(x) = \max(0, x)$$

3.1.5 Pooling Layer

Max pooling reduces spatial dimensions:

$$Y(i, j) = \max_{(m,n)} X(i + m, j + n)$$

3.1.6 CNN Architecture Diagram

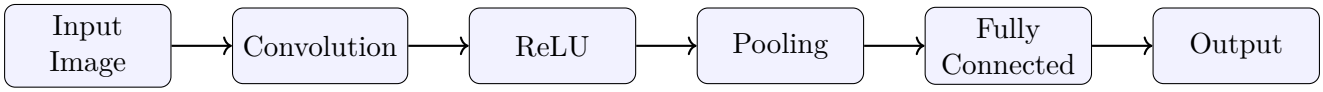


Figure 3.1: Basic Convolutional Neural Network (CNN) Architecture

3.1.7 Types of Gradient Descent

Types of Gradient Descent

Method	**Data Used per Update**
Batch Gradient Descent (Batch GD)	Entire dataset
Stochastic Gradient Descent (SGD)	One sample
Mini-batch Gradient Descent (Mini-batch GD)	Small batch of samples

3.2 Stochastic Gradient Descent (SGD)

3.2.1 Gradient Descent Update Rule

SGD Update

$$\boxed{\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(x_i, y_i)}$$

3.2.2 SGD with Momentum

$$\boxed{v_t = \gamma v_{t-1} + \eta \nabla \mathcal{L}}, \quad \boxed{\theta_{t+1} = \theta_t - v_t}$$

3.3 Batch Normalization

3.3.1 Batch Normalization Steps

Mean:

$$\mu_B = \frac{1}{m} \sum_{i=1}^{m} x_i$$

Variance:

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^{m} (x_i - \mu_B)^2$$

Normalization:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

Scale and Shift:

$$\boxed{y_i = \gamma \hat{x}_i + \beta}$$

3.3.2 Sample Code Template (PyTorch — Jupyter Notebook Style)

Jupyter Notebook – CNN Training with SGD

In [1]: Import Libraries

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
import matplotlib.pyplot as plt
```

In [2]: Load Dataset

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

train_loader = torch.utils.data.DataLoader(
    datasets.MNIST('./data', train=True, download=True,
                   transform=transform),
    batch_size=64, shuffle=True)
```

In [3]: Define CNN Model

```
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, 3)
        self.bn1   = nn.BatchNorm2d(32)
        self.relu  = nn.ReLU()
        self.pool  = nn.MaxPool2d(2)
        self.fc1   = nn.Linear(32*13*13, 10)

    def forward(self, x):
        x = self.pool(self.relu(self.bn1(self.conv1(x))))
        x = x.view(x.size(0), -1)
        return self.fc1(x)
```

In [4]: Training Setup

```
model = SimpleCNN()
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)
```

In [5]: Training Loop

```
losses = []
for epoch in range(5):
    running_loss = 0.0
    for data, target in train_loader:
        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    losses.append(running_loss)
    print(f"Epoch {epoch+1}: Loss={running_loss:.4f}")
```

In [6]: Plot Loss Curve

```
plt.plot(losses)
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training Loss Curve')
plt.show()
```

3.4 Comparative Analysis Tables

3.4.1 CNN vs Fully Connected Neural Networks

Comparison of CNN and Fully Connected Neural Networks

Aspect	**CNN**	**Fully Connected Neural Network**
Parameter Sharing	Yes (shared convolution kernels)	No(independent weights)
Spatial Awareness	Preserves spatial locality	Ignores spatial structure
Number of Parameters	Relatively low	Very high
Performance on Images	Excellent	Poor
Computational Cost	Computationally efficient	Computationally expensive

3.4.2 SGD vs Adam Optimizer

Comparison of SGD and Adam Optimizers

Aspect	SGD	Adam
Learning Rate	Fixed (or scheduled)	Adaptive per parameter
Memory Usage	Low	Higher
Convergence Speed	Moderate	Fast
Stability	Sensitive to tuning	More stable
Common Use	Large datasets	Deep architectures

3.5 Laboratory Exercises

3.5.1 Lab 1: Implementing a CNN for Image Classification

Lab Objective

Students will implement a Convolutional Neural Network (CNN) to classify images and understand the effects of convolutions, activation functions, pooling, and fully connected layers.

Dataset:

- **MNIST**: Handwritten digits (10 classes)
  Source: https://yann.lecun.com/exdb/mnist/
- **CIFAR-10**: 32x32 natural images (10 classes)
  Source: https://www.cs.toronto.edu/~kriz/cifar.html

Tasks:

1. Load and normalize the dataset using PyTorch `DataLoader`.
2. Define a CNN model with:
   - Convolution layers
   - Batch Normalization (optional)
   - ReLU activation
   - Max Pooling
   - Fully connected output layer
3. Train the CNN using Stochastic Gradient Descent (SGD).
4. Plot training loss curves over epochs.
5. Evaluate the accuracy on the test dataset.

Expected Outcomes:

- Students can correctly implement CNN layers and understand the forward pass.
- Ability to observe how batch normalization affects convergence.
- Interpretation of training loss curves and test accuracy.

3.5.2 Lab 2: Comparing Optimizers (SGD vs Adam)

Lab Objective

Students will compare the training performance of two optimizers, SGD and Adam, on the same CNN architecture.

Tasks:

1. Use the CNN implemented in Lab 1.
2. Train the model using SGD optimizer with learning rate 0.01.
3. Train the same model using Adam optimizer with default parameters.
4. Plot and compare the training loss curves for both optimizers.
5. Evaluate the final test accuracy.

Expected Outcomes:

- Students understand differences in convergence speed and stability between SGD and Adam.
- Students can analyze optimizer performance for image classification tasks.

—

3.5.3 Lab 3: Experimenting with Batch Normalization

Lab Objective

Students will study the effect of Batch Normalization on CNN training stability and convergence.

Tasks:

1. Add `nn.BatchNorm2d` layers after each convolution layer in the CNN.
2. Train the network with and without Batch Normalization using SGD.
3. Plot and compare training loss curves.
4. Observe the effect on convergence speed and stability.

Expected Outcomes:

- Students can implement Batch Normalization layers.
- Understand how normalization stabilizes and accelerates training.
- Compare performance metrics with the non-normalized network.

Chapter 4

Advanced Generative Models

4.1 Generative Adversarial Networks (GANs)

Variables Used

Variable	Meaning / Use
G	Generator neural network. Creates synthetic data from random noise.
D	Discriminator neural network. Distinguishes real from fake data.
z	Random noise vector sampled from a prior distribution $p_z(z)$, input to G.
x	Real data sample from true distribution $p_{\text{data}}(x)$.
θ_g	Parameters (weights & biases) of generator G.
θ_d	Parameters (weights & biases) of discriminator D.
$p_z(z)$	Probability distribution of noise vector z (e.g., Gaussian).
$p_{\text{data}}(x)$	True data distribution (unknown).
$V(D, G)$	Minimax objective function of GANs.
$\mathbb{E}_{x \sim p_{\text{data}}(x)}[\cdot]$	Expectation over real data samples x.
$\mathbb{E}_{z \sim p_z(z)}[\cdot]$	Expectation over random noise z.

Overview

Generative Adversarial Networks (GANs) are a class of generative models introduced by Ian Goodfellow in 2014. They consist of two neural networks: a **Generator** G and a **Discriminator** D, which compete in a zero-sum game.

Generator

The generator $G(z; \theta_g)$ takes random noise

$$z \sim p_z(z)$$

and generates synthetic data $G(z)$ resembling real data.

Discriminator

The discriminator $D(x; \theta_d)$ tries to distinguish real data

$$x \sim p_{\text{data}}(x)$$

from fake data $G(z)$.

Objective Function (Minimax Game)

The GAN objective is formulated as:

GAN Objective

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)}\Big[\log D(x)\Big] + \mathbb{E}_{z \sim p_z(z)}\Big[\log(1 - D(G(z)))\Big]$$

Training Process

1. Update D to maximize its ability to classify real vs fake data.
2. Update G to fool D, i.e., minimize $\log(1 - D(G(z)))$.

Variants of GANs

- **DCGAN:** Uses convolutional networks.
- **Conditional GAN (cGAN):** Generates data conditioned on class labels.
- **Wasserstein GAN (WGAN):** Uses Wasserstein distance for more stable training.

4.2 Deterministic and Variational Autoencoders (VAEs)

4.2.1 Deterministic Autoencoder

Deterministic Autoencoder Variables

Variable	Meaning / Use
x	Input data.
$\hat{x}$	Reconstructed input from the decoder.
z	Latent representation in the bottleneck layer.
f_θ	Encoder function mapping $x \to z$.
g_ϕ	Decoder function reconstructing $\hat{x}$ from z.

Encoder:

$$z = f_\theta(x)$$

Decoder:

$$\hat{x} = g_\phi(z)$$

Loss Function (Reconstruction Loss):

Deterministic Autoencoder Loss

$$\mathcal{L}_{AE} = \|x - \hat{x}\|^2$$

4.2.2 Variational Autoencoder (VAE)

VAE Variables

Variable	Meaning / Use
x	Input data.
z	Latent variable sampled from learned distribution $q_\phi(z\|x)$.
$\mu(x), \sigma^2(x)$	Mean and variance predicted by encoder for latent variable z.
$q_\phi(z\|x)$	Encoder distribution over latent variables.
$p_\theta(x\|z)$	Decoder distribution reconstructing x from z.
$\mathcal{L}_{VAE}$	VAE loss (ELBO).
KL	Kullback-Leibler divergence between latent distribution and standard normal.

Encoder:

$$z \sim q_\phi(z|x) = \mathcal{N}(z; \mu(x), \sigma^2(x))$$

Decoder:

$$\hat{x} \sim p_\theta(x|z)$$

Loss Function (ELBO - Evidence Lower Bound):

VAE Loss Function

$$\mathcal{L}_{VAE} = -\mathbb{E}_{z\sim q_\phi(z|x)}\Big[\log p_\theta(x|z)\Big] + KL\Big(q_\phi(z|x)\|p(z)\Big)$$

KL Divergence for Gaussian Latent Variables:

KL Divergence

$$KL(\mathcal{N}(\mu, \sigma^2)\|\mathcal{N}(0,1)) = \frac{1}{2}\sum_{j=1}^{d}\Big(\mu_j^2 + \sigma_j^2 - \log\sigma_j^2 - 1\Big)$$

4.3 Flow-based Models (e.g., PixelCNN, WaveNet)

Flow-based models are **invertible generative models** that map a simple distribution (like Gaussian) to complex data distributions using a sequence of invertible transformations.

Variables Used

Flow-based Model Variables

Variable	Meaning / Use
x	Observed data sample.
z	Latent variable drawn from simple distribution $p_z(z)$.
f_θ	Invertible mapping function that transforms $z \to x$.
$p_X(x)$	Probability density of observed data.
$p_Z(z)$	Probability density of latent variable z (e.g., Gaussian).
$\det \frac{\partial f_\theta^{-1}(x)}{\partial x}$	Determinant of the Jacobian of the inverse transformation.

4.3.1 Mathematical Formulation

Let x be the data and z the latent variable:

$$x = f_\theta(z), \quad z \sim p_z(z)$$

Probability Density Transformation (Change of Variables Formula):

Flow-based Model Density Transformation

$$p_X(x) = p_Z(z) \left| \det \frac{\partial f_\theta^{-1}(x)}{\partial x} \right|$$

Key Examples

- **PixelCNN:** Models image pixels sequentially using autoregressive conditional distributions.
- **WaveNet:** Models audio waveforms sequentially using dilated convolutions.

4.4 Self-Supervised Learning Methods (SSL)

Self-supervised learning (SSL) is a method to learn representations without labeled data by creating artificial labels from the input data itself.

Variables Used

SSL Variables

Variable	Meaning / Use
h_i, h_j, h_k	Representations (embeddings) of input samples.
τ	Temperature parameter controlling sharpness in similarity.
$\text{sim}(\cdot, \cdot)$	Cosine similarity function between embeddings.
L_{InfoNCE}	InfoNCE loss used in contrastive learning.

Example Approaches

1. Contrastive Learning: Maximizes agreement between different views of the same data (positive pairs) and minimizes agreement with other data (negative pairs).

InfoNCE Loss

$$L_{\text{InfoNCE}} = -\log \frac{\exp\left(\text{sim}(h_i, h_j)/\tau\right)}{\sum_{k=1}^{N} \exp\left(\text{sim}(h_i, h_k)/\tau\right)}$$

2. Masked Prediction: Mask part of the input and predict it (e.g., BERT, MAE).

3. Generative Pretext Tasks: Predict rotations, jigsaw patches, or future frames in videos.

Summary Table: Key Comparisons of Generative Models and SSL

Key Comparisons

Model Type	Latent Representation	Loss Function	Key Feature
GAN	Deterministic	Adversarial	Adversarial training
VAE	Probabilistic	ELBO (Recon + KL)	Smooth latent space
Flow-based	Deterministic	Likelihood	Invertible mapping
SSL	Learned features	Contrastive / Pretext	Self-supervised, no labels

Lab Exercise: Advanced Generative Models (Chapter 4)

Prepared for Students

Lab 4.1: Generative Adversarial Networks (GANs)

Objective: Understand GAN architecture and train a simple GAN to generate synthetic images.
Dataset: MNIST handwritten digits
Source: http://yann.lecun.com/exdb/mnist/
Tasks:

1. Load and preprocess MNIST dataset.
2. Build Generator and Discriminator networks.
3. Train GAN using adversarial loss:

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}(x)}[\log D(x)] + \mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))]$$

4. Generate and visualize synthetic images.
5. Plot Generator and Discriminator loss curves.

Python Code Example:

```
from tensorflow.keras.datasets import mnist
import numpy as np

# Load dataset
(x_train, _), (_, _) = mnist.load_data()
x_train = x_train / 255.0
x_train = np.expand_dims(x_train, axis=-1)

# GAN networks would be defined here...
```

Expected Outcome: Students can generate MNIST-like images and understand GAN training dynamics.

Lab 4.2: Variational Autoencoders (VAEs)

Objective: Implement a VAE for image reconstruction and generation, visualize latent space.
Dataset: Fashion-MNIST
Source: https://github.com/zalandoresearch/fashion-mnist
Tasks:

1. Build encoder to output $\mu(x)$ and $\sigma^2(x)$.
2. Sample latent variable $z = \mu + \sigma \odot \epsilon$, $\epsilon \sim \mathcal{N}(0, 1)$.
3. Build decoder to reconstruct x.
4. Train with ELBO loss:

$$\mathcal{L}_{\text{VAE}} = -\mathbb{E}_{z \sim q_\phi(z|x)}[\log p_\theta(x|z)] + \text{KL}(q_\phi(z|x) \| p(z))$$

5. Generate images from $z \sim \mathcal{N}(0, 1)$.
6. Visualize latent space using PCA or t-SNE.

Python Code Example:

```
from tensorflow.keras.datasets import fashion_mnist

(x_train, _), (_, _) = fashion_mnist.load_data()
x_train = x_train / 255.0
x_train = np.expand_dims(x_train, axis=-1)

# Encoder, decoder, and VAE training code here...
```

Expected Outcome: Generate realistic images and understand latent space representation.

Lab 4.3: Flow-based Models

Objective: Implement a simple flow-based model for density estimation, understand invertible transformations.
Dataset: CIFAR-10
Source: https://www.cs.toronto.edu/~kriz/cifar.html
Tasks:

1. Implement Normalizing Flow using affine coupling layers.
2. Transform latent Gaussian $z \sim \mathcal{N}(0, 1)$ to data distribution $x = f_\theta(z)$.
3. Compute log-likelihood:

$$\log p_X(x) = \log p_Z(z) + \log \left| \det \frac{\partial f_\theta^{-1}(x)}{\partial x} \right|$$

4. Train model to maximize likelihood.
5. Generate new samples and visualize.

Python Code Example:

```
from tensorflow.keras.datasets import cifar10

(x_train, _), (_, _) = cifar10.load_data()
x_train = x_train / 255.0

# Normalizing Flow model implementation here...
```

Expected Outcome: Generate data and understand flow-based likelihood estimation.

Lab 4.4: Self-Supervised Learning (SSL)

Objective: Learn representations without labels, implement contrastive learning.
Dataset: CIFAR-10 (unlabeled)
Source: https://www.cs.toronto.edu/~kriz/cifar.html
Tasks:

1. Create augmented image pairs (crop, rotation, flip).
2. Pass images through encoder to get embeddings h_i and h_j.
3. Train using InfoNCE loss:

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp(\text{sim}(h_i, h_j)/\tau)}{\sum_{k=1}^{N} \exp(\text{sim}(h_i, h_k)/\tau)}$$

4. Visualize embeddings with t-SNE.
5. Optional: Use embeddings for downstream classification.

Python Code Example:

```
# Using CIFAR-10 without labels
from tensorflow.keras.datasets import cifar10

(x_train, _), (_, _) = cifar10.load_data()
x_train = x_train / 255.0

# Data augmentation and contrastive learning implementation
    here...
```

Expected Outcome: Learn meaningful features and visualize feature space.

Lab Report Deliverables

1. Clean, documented Python code notebook.
2. Generated samples (images or visualizations).
3. Loss curves for training.
4. Discussion and analysis of results.
5. Include dataset sources for reproducibility.

Chapter 5

Sequential Models

Sequential models are designed to handle data with temporal or sequential dependencies, such as time series, text, speech, and video. Unlike traditional feedforward neural networks, sequential models maintain a memory of past inputs to influence current predictions.

5.1 Recurrent Neural Networks (RNNs)

Concept

RNNs are neural networks that process sequences by maintaining a hidden state h_t that captures information about previous inputs. They are especially suited for sequential tasks like language modeling, machine translation, and time-series prediction.
Input sequence: $x_1, x_2, \ldots, x_T$
Output sequence: $y_1, y_2, \ldots, y_T$
Hidden state: h_t (represents memory at time t)

Mathematical Formulation

For time step t:

$$h_t = \phi(W_{xh}x_t + W_{hh}h_{t-1} + b_h), \quad y_t = W_{hy}h_t + b_y$$

Where:

$x_t \in \mathbb{R}^{n_x} \rightarrow$ input vector
$h_t \in \mathbb{R}^{n_h} \rightarrow$ hidden state
$y_t \in \mathbb{R}^{n_y} \rightarrow$ output
$W_{xh}, W_{hh}, W_{hy} \rightarrow$ weight matrices
$b_h, b_y \rightarrow$bias vectors
ϕ activation function (usually tanh or ReLU)

Challenges

- **Vanishing gradient:** Gradients become very small → difficulty learning long-term dependencies.
- **Exploding gradient:** Gradients become very large → unstable training.

5.2 Long Short-Term Memory (LSTM) Networks

Concept

LSTMs are a special type of RNN designed to avoid vanishing gradients and capture long-term dependencies. They introduce gates to control the flow of information:

- **Forget gate** $f_t \rightarrow$ decides what information to discard.
- **Input gate** $i_t \rightarrow$ decides which new information to store.
- **Cell state** $C_t \rightarrow$ memory of the network.
- **Output gate** $o_t \rightarrow$ decides what part of the cell state to output.

Mathematical Formulation

$$
\begin{aligned}
f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f) \\
i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i) \\
\tilde{C}_t &= \tanh(W_C x_t + U_C h_{t-1} + b_C) \\
C_t &= f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \\
o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o) \\
h_t &= o_t \odot \tanh(C_t)
\end{aligned}
$$

Where:

$\sigma \rightarrow$ sigmoid activation

$\odot \rightarrow$ element-wise multiplication

$C_t \rightarrow$ cell state

$h_t \rightarrow$ hidden state (output)

Key Advantage

LSTMs can retain information over long sequences due to the cell state C_t and gating mechanisms.

5.3 Gated Recurrent Units (GRUs)

Concept

GRUs are a simplified variant of LSTMs. They combine the forget and input gates into a single update gate and remove the separate cell state, using only the hidden state. GRUs are faster and easier to train while still handling long-term dependencies.

Mathematical Formulation

$$
\begin{aligned}
z_t &= \sigma(W_z x_t + U_z h_{t-1} + b_z) && \text{(update gate)} \\
r_t &= \sigma(W_r x_t + U_r h_{t-1} + b_r) && \text{(reset gate)} \\
\tilde{h}_t &= \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h) && \text{(candidate state)} \\
h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t
\end{aligned}
$$

Where:

$z_t \rightarrow$ controls how much past information to keep

$r_t \rightarrow$ controls how much past information to forget when computing new candidate

$h_t \rightarrow$ hidden state (acts as memory)

Key Difference

GRUs have fewer parameters and merge cell and hidden states, making them computationally lighter compared to LSTMs.

Summary of RNN Variants

Model	Memory Mechanism	Gates	Pros	Cons
Vanilla RNN	Hidden state h_t	None	Simple, interpretable	Vanishing/exploding gradient
LSTM	Hidden + cell state h_t, C_t	Input, Forget, Output	Handles long sequences, avoids vanishing gradient	More parameters, slower
GRU	Hidden state h_t	Update, Reset	Efficient, fewer parameters	Slightly less expressive than LSTM

Laboratory Exercise: Sequential Models (RNN

Lab Objectives:

- Implement RNN, LSTM, and GRU networks for sequence prediction.
- Train models on a time-series dataset and compare performance.
- Visualize predicted vs actual sequences.
- Analyze the impact of sequence length, hidden units, and learning rate.

Problem Statement

Predict the next day's temperature from the previous 30 days using sequential models:

- Dataset: Daily minimum temperatures in Melbourne (1981-1990)
- Source: Kaggle: Daily Minimum Temperatures
- Task: Compare RNN, LSTM, and GRU for sequence prediction.

Tasks

Task 1: Data Preprocessing

1. Load dataset and parse `Date`.
2. Normalize temperatures using Min-Max scaling.
3. Create input sequences of 30 days to predict the next day.
4. Split dataset: 80% train, 20% test.

Task 2: Implement RNN

1. Build a SimpleRNN model.
2. Train to predict the next-day temperature.
3. Plot training and validation loss.

Task 3: Implement LSTM

1. Replace RNN with an LSTM layer.
2. Train and plot loss curves.

Task 4: Implement GRU

1. Replace RNN/LSTM layer with GRU.
2. Train and plot loss curves.

Task 5: Model Evaluation

1. Predict test set temperatures.
2. Compute Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$$

3. Plot actual vs predicted values.
4. Compare model performance and discuss.

Python Skeleton Code

```python
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, LSTM, GRU, Dense

# Load dataset
data = pd.read_csv('daily-min-temperatures.csv', parse_dates=['Date'], index_col='Date')
temps = data['Temp'].values.reshape(-1,1)

# Normalize
scaler = MinMaxScaler()
temps_scaled = scaler.fit_transform(temps)

# Create sequences
def create_sequences(data, seq_length):
    X, y = [], []
    for i in range(len(data)-seq_length):
        X.append(data[i:i+seq_length])
        y.append(data[i+seq_length])
    return np.array(X), np.array(y)

SEQ_LEN = 30
X, y = create_sequences(temps_scaled, SEQ_LEN)

# Train-test split
split = int(0.8*len(X))
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# Reshape for RNN input (samples, timesteps, features)
X_train = X_train.reshape((X_train.shape[0], SEQ_LEN, 1))
X_test = X_test.reshape((X_test.shape[0], SEQ_LEN, 1))

# Build LSTM model (example)
model = Sequential()
model.add(LSTM(50, activation='tanh', input_shape=(SEQ_LEN,1)))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mse')

# Train model
history = model.fit(X_train, y_train, epochs=50, batch_size=32, validation_split=0.1)

# Plot training loss
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.show()
```

Optional Exploration

- Change sequence length (e.g., 10, 20, 50) and compare performance.
- Vary number of hidden units.
- Add dropout layers to prevent overfitting.

Expected Outcomes

- Students will implement RNN, LSTM, and GRU networks.
- LSTM and GRU will handle long-term dependencies better than vanilla RNN.
- Students gain hands-on experience with sequence prediction.
- Visualization of predicted vs actual sequences aids model understanding.

Chapter 6

Learning Theory and Challenges

Introduction

Learning theory explains why machine learning algorithms work, their limitations, and the challenges faced when learning from real-world data.

6.1 Theoretical Foundations of Learning

Learning theory studies the relationship between data, models, and generalization performance.

Learning Paradigm

Basic definitions:

- Input space: $\mathcal{X}$ (set of input features)
- Output space: $\mathcal{Y}$ (set of target labels)
- Data distribution: $\mathcal{D}$ (unknown true distribution)
- Training dataset:
$$S = \{(x_i, y_i)\}_{i=1}^{n} \sim \mathcal{D}$$
where n is the number of samples.
- Hypothesis space:
$$\mathcal{H} = \{h : \mathcal{X} \to \mathcal{Y}\}$$

The learning objective is:

$$h^* = \arg\min_{h \in \mathcal{H}} L_{\mathcal{D}}(h)$$

where: h^* is the optimal model, h is a hypothesis, and $L_{\mathcal{D}}(h)$ is the expected loss over distribution $\mathcal{D}$.

Empirical Risk Minimization (ERM)

True risk:

$$R(h) = \mathbb{E}_{(x,y)\sim\mathcal{D}}[\ell(h(x), y)]$$

where: $\ell(\cdot)$ is the loss function, (x, y) is a data point.

Empirical risk:

$$\hat{R}(h) = \frac{1}{n}\sum_{i=1}^{n} \ell(h(x_i), y_i)$$

where: (x_i, y_i) are training samples and n is the dataset size.

ERM principle:

$$\hat{h} = \arg\min_{h \in \mathcal{H}} \hat{R}(h)$$

Bias Variance Tradeoff

The expected prediction error is:

$$\mathbb{E}[(y - \hat{f}(x))^2] = \text{Bias}^2 + \text{Variance} + \sigma^2$$

where: y is the true output, $\hat{f}(x)$ is the predicted output, and σ^2 is irreducible noise.

- High bias $\rightarrow$ underfitting
- High variance $\rightarrow$ overfitting

Overfitting and Underfitting

Overfitting:

$$\hat{R}(h) \ll R(h)$$

where: training error is much smaller than true error.

Underfitting:

$$\hat{R}(h) \text{ is high and } R(h) \text{ is high}$$

where: the model is too simple.

Regularization

Regularization adds a penalty to reduce model complexity.

Ridge Regression (L2):

$$\text{Cost} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda\sum_{i=1}^{m} w_i^2$$

where: y_i is the true output, $\hat{y}_i$ is the predicted output, w_i are model weights, m is the number of weights, n is the number of samples, and λ controls regularization strength.

Lasso Regression (L1):

$$\text{Cost} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda\sum_{i=1}^{m} |w_i|$$

where: $|w_i|$ is the absolute value of weight w_i, encouraging sparsity.

Generalization Theory

The generalization gap is:

$$|R(h) - \hat{R}(h)|$$

where: smaller values indicate better generalization.

Vapnik-Chervonenkis(VC) Dimension

Vapnik-Chervonenkis (VC) dimension is a measure of the capacity of a hypothesis set to fit different data sets.

$$\text{VC}(\mathcal{H}) = \max n \text{ such that } \mathcal{H} \text{ can shatter } n \text{ points}$$

A simple generalization bound:

$$R(h) \leq \hat{R}(h) + \sqrt{\frac{\text{VC}(\mathcal{H})\log n}{n}}$$

where: the second term bounds the generalization error.

Practical Lab Exercise

Lab Overview

This laboratory exercise provides a unified, hands-on understanding of learning theory, data bias, and meta-learning using a real-world dataset. Students will observe generalization behavior, identify bias, and simulate learning-to-learn concepts using simple supervised models.

Lab Objectives

After completing this lab, students will be able to:

- Explain generalization and overfitting using empirical results.
- Identify and mitigate data bias in machine learning models.
- Demonstrate the basic idea of meta-learning (learning to learn).

Software and Tools Required

- Python 3.x
- Jupyter Notebook
- Libraries: numpy, pandas, matplotlib, scikit-learn

Dataset Description and Source

Adult Income Dataset

Source: UCI Machine Learning Repository **Link:** https://archive.ics.uci.edu/ml/datasets/adult

The dataset contains demographic information such as age, education, occupation, gender, and race. The task is to predict whether a person earns more than $50K per year$.

Part A: Theoretical Foundations of Learning

Objective

To understand generalization, empirical risk minimization, and overfitting by comparing training and test performance.

- Split the dataset into training and testing sets.
- Train a Logistic Regression model.
- Train a deep Decision Tree model.
- Compare training and test accuracy.

Key Formula

[Generalization Gap = Training Error - Test Error]

Part B: Addressing Data Bias in Models

Objective

To identify bias in predictions across demographic groups and apply a basic mitigation technique.

- Evaluate model accuracy for male and female groups separately.
- Measure disparity in performance.
- Apply random undersampling to balance the dataset.
- Retrain the model and re-evaluate fairness.

Bias Metric

$[\text{ Bias Gap } = |\text{Accuracy}_{Male} - Accuracy_{Female}|]$

Part C: Concepts of Meta-Learning (Learning to Learn)

Objective

To demonstrate how prior learning can help a model adapt faster to new but related tasks.

- Create multiple learning tasks by splitting data based on education level.
- Train a base Logistic Regression model.
- Fine-tune the model on a new task using fewer samples.
- Compare convergence and performance.

Meta-Learning Update Rule

$[\theta_{new} = \theta_{old} - \alpha \nabla L_{task}]$

Student Reflection Questions

1. Why does the decision tree overfit the training data?
2. How did bias appear in the dataset?
3. Which mitigation strategy improved fairness?
4. Why does meta-learning require fewer samples?
5. How do these challenges appear in real-world AI systems?